

## GUÍA 13: Integrando Recurso Nativo Camara

| Sigla   | Asignatura                         | Experiencia de Aprendizaje                                                                  |
|---------|------------------------------------|---------------------------------------------------------------------------------------------|
| DSY1105 | Desarrollo de Aplicaciones Móviles | EA2: Desarrollando componentes visuales y funcionalidades nativas para aplicaciones móviles |
| Tiempo  | Modalidad de Trabajo               | Indicadores de logro                                                                        |
| 1 hora  | Parejas                            | IL2.1, IL2.2, IL2.3 y IL2.4                                                                 |



### Antecedentes generales

Esta actividad busca que integres recursos nativos avanzados del dispositivo móvil, como la galería y la cámara, utilizando Jetpack Compose, Material 3, la arquitectura MVVM y el API moderno `ActivityResultLauncher`. Su propósito es que comprendas cómo acceder y gestionar funcionalidades del sistema operativo Android de forma segura y estructurada, incorporando estos recursos en una interfaz declarativa y reactiva que mantenga la coherencia visual y funcional de la aplicación.



### Requerimientos para esta actividad

Para el desarrollo de esta actividad deberás disponer de:

- Un proyecto Android Studio nuevo (API 33 o superior)
- Arquitectura MVVM con estructura de carpetas definida
- Jetpack Compose y Material 3 actualizados
- Permisos declarados en `AndroidManifest.xml`.
- Dependencias actualizadas
- Conexión a internet para consultar documentación oficial
- Dispositivo Móvil Android
- Un repositorio GitHub activo y funcional
- Tablero Trello para registrar la tarea por pareja



## Actividad

En esta actividad práctica, desarrollarán un flujo completo de selección y visualización de imágenes en una app de perfil de usuario, aplicando los principios del patrón arquitectónico MVVM y aprovechando la reactividad de StateFlow en Jetpack Compose.

A través de una pantalla de perfil, se espera que integren dos recursos nativos del dispositivo Android: la galería y la cámara. El objetivo es que puedan capturar una imagen nueva o seleccionar una ya existente, y visualizarla en la interfaz mediante un componente @Composable reutilizable, manteniendo el estado sincronizado con un ViewModel compartido.

La tarea busca que exploren de manera autónoma la correcta definición de modelos de estado (Uri?), la manipulación del estado con MutableStateFlow, y la observación reactiva con collectAsState. Además, deberán aplicar principios de jerarquía visual, diseño accesible con Material 3, y una organización lógica del código que asegure la separación de responsabilidades en capas (UI, ViewModel, almacenamiento temporal con FileProvider).

El trabajo se desarrollará en parejas, registrando los avances en GitHub (commits, rama feature/imagen-inteligente) y Trello (tarjeta: Imagen desde cámara y galería). Deberá presentarse como evidencia el correcto funcionamiento de ambos lanzadores (TakePicture() y GetContent()), el componente visual, la persistencia del estado y la ejecución real en un dispositivo físico.

### ◇ Parte 1 – Configurar permisos

- Declarar permisos para acceder a la galería y la cámara en el archivo AndroidManifest.xml.
- Configurar el acceso a almacenamiento temporal mediante FileProvider para usar la cámara.

### ◇ Parte 2 – Crear el ViewModel de perfil

- Declarar una variable para guardar el URI de la imagen seleccionada o capturada.
- Crear funciones para actualizar la imagen desde la galería y desde la cámara.
- Usar StateFlow para mantener el estado reactivo en la interfaz.

### ◇ Parte 3 – Crear componente ImagenInteligente

- Crear un archivo ImagenInteligente.kt dentro de una carpeta components.
- El componente debe mostrar la imagen en forma circular.
- Si no hay imagen, mostrar un ícono de perfil por defecto.

### ◇ Parte 4 – Crear la pantalla PerfilScreen

- Mostrar el componente ImagenInteligente al centro de la pantalla.
- Crear dos botones:
  - Uno para abrir la galería y seleccionar una imagen.
  - Otro para activar la cámara y capturar una imagen nueva.
- Usar rememberLauncherForActivityResult para manejar ambos casos.
- Enviar el resultado al ViewModel.

### ◇ Parte 5 – Comprobar en dispositivo físico

- Ejecutar la app en un teléfono real.



- Probar primero la galería: seleccionar una imagen y verificar que se visualice.
- Luego usar la cámara: capturar una imagen, aceptar y confirmar que se muestra correctamente.
- Verificar que no crashee si se cancela la acción.

◇ **Parte 6 – Evidencia de trabajo colaborativo**

- Subir un commit con el mensaje: "Funcionalidad cámara y galería integrada".
- Rama: feature/imagen-inteligente
- Subir dos capturas:
  - Una con imagen seleccionada desde galería.
  - Otra con imagen capturada desde cámara.
- Crear tarjeta Trello: Cámara y galería
  - Checklist: ☐ ViewModel, ☐ ImagenInteligente, ☐ Galería, ☐ Cámara, ☐ Test en físico, ☐ GitHub, ☐ Evidencia.